Calliope: A TTS-based Narrated E-book Creator Ensuring Exact Synchronization, Privacy, and Layout Fidelity

Hugo L. Hammer[1,2], Vajira Thambawita[2], and Pål Halvorsen[2,1]

[1]Oslo Metropolitan University, Oslo, Norway
[2]SimulaMet, Oslo, Norway

Abstract

A narrated e-book combines synchronized audio with digital text, highlighting the currently spoken word or sentence during playback. This format supports early literacy and assists individuals with reading challenges, while also allowing general readers to seamlessly switch between reading and listening. With the emergence of natural-sounding neural Text-to-Speech (TTS) technology, several commercial services have been developed to leverage these technology for converting standard text e-books into high-quality narrated e-books. However, no open-source solutions currently exist to perform this task. In this paper, we present **Calliope**, an open-source framework designed to fill this gap. Our method leverages state-of-the-art open-source TTS to convert a text e-book into a narrated e-book in the EPUB 3 Media Overlay format. The method offers several innovative steps: audio timestamps are captured directly during TTS, ensuring exact synchronization between narration and text highlighting; the publisher's original typography, styling, and embedded media are strictly preserved; and the entire pipeline operates offline. This offline capability eliminates recurring API costs, mitigates privacy concerns, and avoids copyright compliance issues associated with cloud-based services. The framework currently supports the state-of-the-art open-source TTS systems XTTS-v2 and Chatterbox. A potential alternative approach involves first generating narration via TTS and subsequently synchronizing it with the text using forced alignment. However, while our method ensures exact synchronization, our experiments show that forced alignment introduces drift between the audio and text highlighting significant enough to degrade the reading experience. Source code and usage instructions are available at `https://github.com/hugohammer/TTS-Narrated-Ebook-Creator.git`.

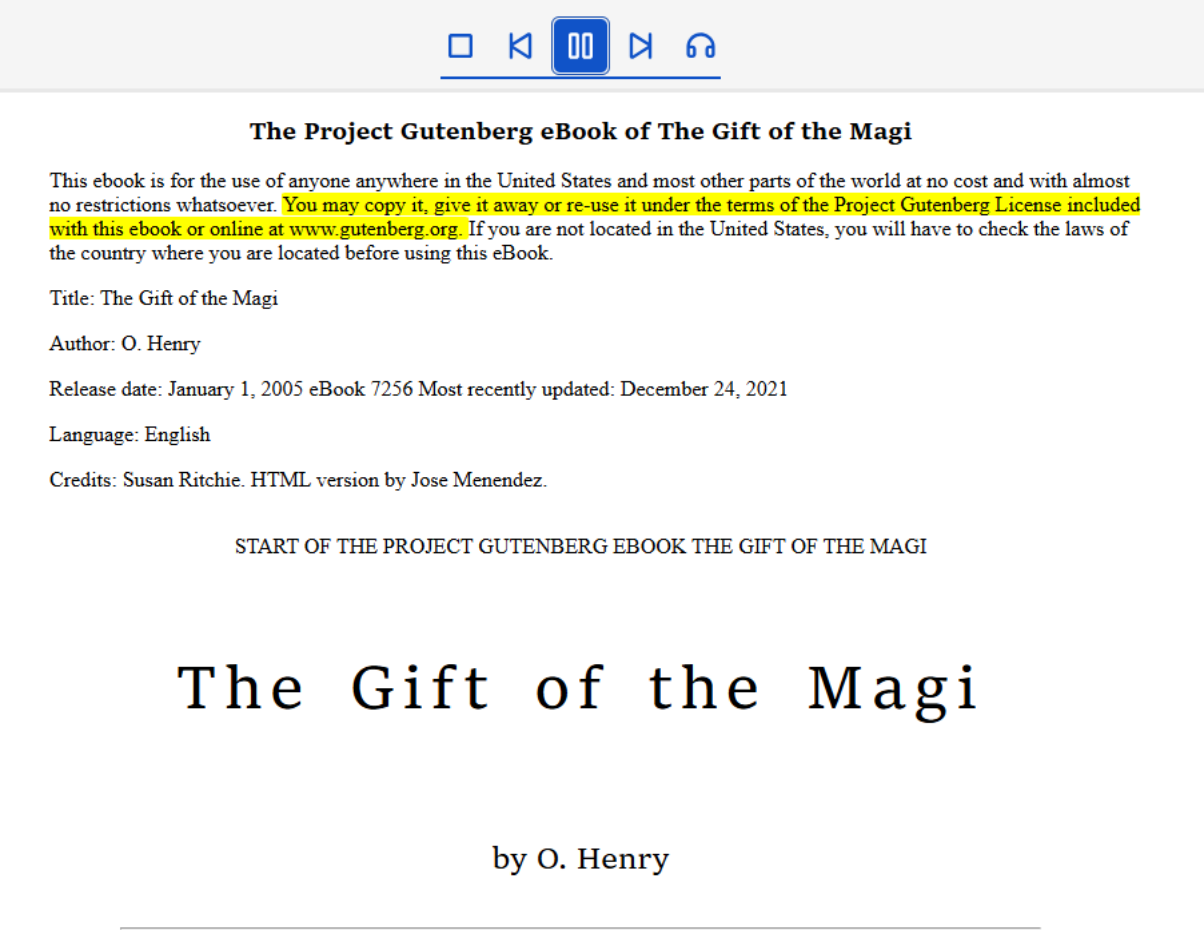
Keywords: narrated e-book, neural networks, text-to-speech

1 Introduction

"Narrated E-book", also referred to as "Read Aloud" or "Immersive reading", refers to technology that reads text aloud while highlighting the word or sentence currently being spoken. This feature supports early readers in developing literacy skills and assists individuals with dyslexia or other reading challenges [9]. It also benefits general readers by enabling seamless switching between reading and listening, depending on their preference at any given moment.

Several technologies enable synchronized text and audio playback. A key open-source standard is EPUB 3 Media Overlays, which uses SMIL (Synchronized Multimedia Integration Language) to align audio narration with text content [22]. Another major standard is DAISY (Digital Accessible Information System), designed primarily for individuals with print disabilities [4]. EPUB 3 Media Overlays modernize and generalize DAISY concepts. For web-based real-time applications, the W3C Web Speech API allows developers to implement text-to-speech (TTS) and speech recognition directly in browsers, supporting text highlighting during speech synthesis [12].

Recent years have seen the emergence of natural-sounding neural Text-to-Speech (TTS) technology. Several commercial services, such as ElevenReader and NaturalReader, leverage this technology to



The Project Gutenberg eBook of The Gift of the Magi

This ebook is for the use of anyone anywhere in the United States and most other parts of the world at no cost and with almost no restrictions whatsoever. You may copy it, give it away or re-use it under the terms of the Project Gutenberg License included with this ebook or online at www.gutenberg.org. If you are not located in the United States, you will have to check the laws of the country where you are located before using this eBook.

Title: The Gift of the Magi

Author: O. Henry

Release date: January 1, 2005 eBook 7256 Most recently updated: December 24, 2021

Language: English

Credits: Susan Ritchie. HTML version by Jose Menendez.

START OF THE PROJECT GUTENBERG EBOOK THE GIFT OF THE MAGI

The Gift of the Magi

by O. Henry

Figure 1: Example of an EPUB 3 Media Overlay file created using the proposed method. The figure demonstrates active text highlighting synchronized with audio playback, while the publisher's original layout and styling are preserved.

convert standard text e-books into high-quality narrated e-books [13, 6]. However, currently, no open-source solutions exist to perform this task. To address this gap, we present **Calliope**, an open-source framework designed to fill this void. Our method converts an e-book in the EPUB format into a narrated e-book in the EPUB 3 Media Overlay format, obtaining audio via state-of-the-art (SOTA) open-source TTS models. The implementation supports both XTTS-v2 [2] and Chatterbox [19] for audio generation. Calliope incorporates several innovative steps to ensure high quality: audio timestamps are computed directly during the TTS process, ensuring exact synchronization between text highlighting and audio; the pipeline performs surgical modifications of the EPUB container files, preserving the publisher's original typography, styling, and embedded media; and it operates entirely offline, ensuring user privacy and maintaining strict control over copyrighted material. Figure 1 illustrates an example of a resulting EPUB 3 Media Overlay file created using our methodology when played in the Thorium reader*[5]. As shown, the text is highlighted (yellow) and synchronized with the audio, while the publisher's original layout and styling remain preserved.

Several related open-source or partly open-source solutions exist to facilitate the creation of narrated e-books. Solutions such as *Storyteller* [21] and *Syncabook* [16] function as forced alignment engines, requiring pre-existing audiobook files and using algorithms such as Dynamic Time Warping (DTW) to map the audio stream to the text [20, 15]. Forced alignment tools are prone to synchronization drift, resulting in misalignment between the highlighting and the audio. In some cases, this leads to audio truncation or, depending on the reader implementation, time-stretching artifacts where the audio sounds

*A short video demonstration is available at `https://youtu.be/j15BHY1hh7w`.

distorted. In principle, one could generate audio using TTS and then apply forced alignment, but the experiments in this paper show that forced alignment results in drift between audio and text highlighting significant enough to reduce the reading experience. *Epub2tts* [1] leverages SOTA TTS models, such as Chatterbox, but does not create narrated e-books, only audiobooks. Finally, *Audible-epub3-maker* [7] generates synchronized EPUB 3 Media Overlays but relies on commercial cloud APIs. This reliance introduces recurring costs and necessitates the uploading of sensitive or copyrighted material to third-party servers. Furthermore, existing open-source pipelines employ a "reconstruction" strategy that strips the original CSS and layout, degrading the visual fidelity of the resulting narrated e-book. Table 1 provides an overview of the main properties of our proposed method (first row) compared to existing open-source solutions.

Table 1: Comparison with existing open-source solutions.

Method	**Sync Strategy**	**Privacy**	**Layout**	**Cost**	**Sync. Accuracy**
Calliope	Deterministic	Local	Preserved	Free	Exact
Syncabook [16]	Forced alignment	Local	Reconstructed	Free	Variable
Storyteller [21]	Transcribe + alignment	Local	Reconstructed	Free	Variable
Epub2tts [1]	Audio only	Local	N/A	Free	N/A
Audible-maker [7]	Cloud TTS	Cloud	Reconstructed	Paid	Unknown

The main contributions of this paper are:

- We present an open-source, offline solution for creating EPUB 3 files with Media Overlays using TTS.
- We employ two state-of-the-art open-source TTS methods: XTTS-v2 [2] and Chatterbox [19].
- The offline architecture preserves user privacy and mitigates copyright risks associated with cloud-based services.
- The solution performs surgical modifications to the source file, ensuring preservation of the publisher's original layout and styling.
- Synchronization timings are computed directly during TTS generation, guaranteeing perfect alignment between text highlighting and audio.
- We introduce a recursive text segmentation algorithm that mitigates the fixed context-window constraints of Transformer-based TTS models, enabling seamless processing of complex sentence structures without truncation or hallucination.

2 The EPUB 3 Standard

An EPUB publication is fundamentally a zipped container that follows the Open Container Format (OCF) [22]. Three key components within the EPUB container are:

1. **Package Document (OPF):** This XML file serves as the central "brain" of the publication. It contains the *Metadata* (bibliographic information), the *Manifest* (a complete inventory of all files in the archive), and the *Spine* (the linear reading order of the XHTML content documents).
2. **Content Documents (XHTML):** These files contain the narrative text and structural markup (HTML5). They also reference auxiliary resources such as images and stylesheets.

3. **Style Sheets (CSS):** These define the visual presentation (fonts, margins, colors) and are linked via standard HTML `<link>` tags within the XHTML headers.

To transform a static e-book into a synchronized audiobook, EPUB 3 introduces *Media Overlays* [22]. This feature establishes a triangulation of references between the text, the audio, and a synchronization map, as described below.

2.1 The Content Document (XHTML)

The text content is segmented, with each synchronization unit (sentence or word) wrapped in a `<span>` element containing a unique identifier (`id`). This identifier serves as the anchor for the synchronization logic:

```
<?xml version="1.0" encoding="UTF-8"?>
<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:epub="http://www.idpf.org/2007/ops"
    lang="en">
  <body>
    <p>
      <span id="c01_s001">Call me Ishmael. </span>
      <span id="c01_s002">Some years ago...</span>
    </p>
  </body>
</html>
```

2.2 The Media Overlay Document (SMIL)

A SMIL file acts as a temporal map that binds a specific XHTML content document to a corresponding audio file. For each XHTML chapter, a corresponding SMIL file is generated mapping the structural IDs (id) to the calculated audio time intervals $[t_{\text{start}}, t_{\text{end}}]$. The references are constructed using relative paths to ensure compatibility across reading systems. The `<seq>` element points to the target XHTML file via the `epub:textref` attribute, and `<par>` elements group the text anchor with the audio playback window:

```
<smil xmlns="http://www.w3.org/ns/SMIL" version="3.0"
    xmlns:epub="http://www.idpf.org/2007/ops">
  <body>
    <seq id="seq1" epub:textref="../text/chapter1.xhtml">
      <par id="par_c01_s001">
        <text src="../text/chapter1.xhtml#c01_s001"/>
        <audio src="../audio/chapter1.mp3"
             clipBegin="0:00:00.000"
             clipEnd="0:00:02.540"/>
      </par>
    </seq>
  </body>
</smil>
```

2.3 The Package Document (OPF)

Finally, the OPF manifest binds these files together. The XHTML item entry includes a `media-overlay` attribute that points to the ID of the SMIL item. The OPF informs the reading system that when the user opens `chapter1.xhtml`, it should simultaneously load `chapter1.smil` for playback:

```
<package ... version="3.0">
  <metadata>
    <meta property="media:duration"
    refines="#smil_01">0:00:15.0</meta>
  </metadata>
  <manifest>
    <item id="text_01" href="text/chapter1.xhtml"
         media-type="application/xhtml+xml"
         media-overlay="smil_01"/>

    <item id="smil_01" href="smil/chapter1.smil"
         media-type="application/smil+xml"/>

    <item id="audio_01" href="audio/chapter1.mp3"
         media-type="audio/mpeg"/>
  </manifest>
</package>
```

3 Neural Text-to-Speech Engines

Our framework supports two distinct state-of-the-art neural architectures for waveform synthesis, namely XTTS-v2 and Chatterbox.

3.1 XTTS-v2 (Coqui):

XTTS-v2 is an autoregressive transformer model based on a GPT-2 architecture. It predicts discrete audio tokens conditioned on input text and a reference speaker embedding. These tokens are converted into a continuous waveform using a HiFi-GAN-based decoder adapted for XTTS [10]. As an autoregressive model, it generates audio sequentially, enabling state-of-the-art zero-shot voice cloning and cross-lingual prosody transfer. However, this sequential dependency limits parallelization, resulting in higher computational latency.

3.2 Chatterbox (Resemble AI):

Chatterbox employs a non-autoregressive Flow Matching architecture built on a 0.5B-parameter Llama transformer backbone. Unlike diffusion models that iteratively denoise signals, Flow Matching learns a continuous probability flow and solves it using an Ordinary Differential Equation (ODE) solver [11]. This design enables stable, high-fidelity synthesis with fewer inference steps compared to traditional diffusion approaches.

4 Methodology

In this section, we describe the proposed framework for converting a static EPUB file into an EPUB 3 with Media Overlays using Neural TTS. The pipeline consists of three distinct phases: (1) text extraction and structural preservation of the original source, (2) TTS synthesis, audio processing, and temporal synchronization (SMIL), and (3) packaging of the final EPUB 3 container. Figure 2 provides a short overview of the three phases.

4.1 Phase 1: Text Extraction and Layout Preservation

Let D represent the Document Object Model (DOM) of an XHTML chapter file. We traverse D to identify the set of block-level elements $\mathcal{B} = \{b_1, b_2, \ldots, b_n\}$ (e.g., paragraphs, headers) that act as leaf

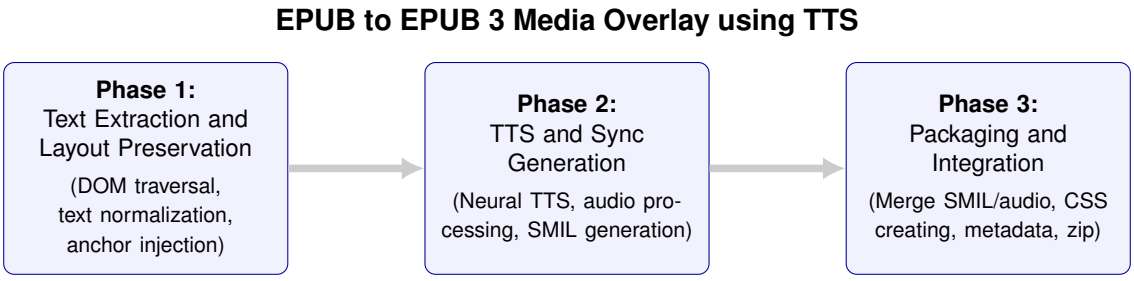


Figure 2: Overview of the three phases of the Calliope methodology.

nodes containing narrative text.

For each block b_j, the raw text content is extracted and normalized to resolve Unicode canonicalization and standardize punctuation. This normalized text is then segmented into an ordered sequence of sentences $S_j = \{s_{j,1}, s_{j,2}, \ldots, s_{j,N_j}\}$ using a standard sentence tokenizer.

This segmentation S_j serves a dual purpose: it defines the input text segments for the TTS engine in Step 2, and simultaneously establishes the structural anchors required for synchronized text highlighting. For each sentence, a structural anchor is injected into the DOM in the form of a `span` element with a globally unique identifier, $id_{j,k}$

$$b'_j = \bigcup_{k=1}^{N_j} \langle \text{span id} = id_{j,k} \rangle s_{j,k} \langle / \text{span} \rangle \tag{1}$$

These insertions occur strictly within the blocks b_j, preserving the parent element's CSS attributes while creating the precise granularity required for sentence-level highlighting.

4.2 Phase 2: TTS and Audio-Text Synchronization

Let $\mathcal{M}$ denote a neural TTS model. For each sentence $s_{j,k}$, the model generates a discrete audio waveform $w_{j,k}$

$$w_{j,k} = \mathcal{M}(s_{j,k}, \theta_{voice}) \tag{2}$$

where θ_{voice} represents the reference speaker embedding, typically computed from a short audio sample. Neural TTS models operate under strict context window constraints; a failure occurs if either the input text sequence or the generated latent sequence exceeds the model's capacity. To guarantee execution stability, we implement a two-tier handling strategy:

1. Pre-emptive Heuristic (Input Constraint): Let $|s_{j,k}|$ denote the character length of sentence $s_{j,k}$. If $|s_{j,k}|$ exceeds a conservative safety threshold λ^+ (e.g., 200 characters), the text is recursively split at the nearest whitespace median. Additionally, certain diffusion-based architectures, such as Chatterbox [19], may degrade on extremely short inputs. To mitigate this, adjacent sentences are merged if $|s_{j,k}|$ falls below a minimum threshold λ^- (e.g., 60 characters).

2. Reactive Error Handling (Output Constraint): Certain text segments possess a high phonetic density relative to their character count. For instance, numerical years (e.g., "1997") expand significantly in token space, potentially causing the decoder to exceed its maximum limit even when the character count is low. Predicting the exact token yield of a text segment is non-trivial. To address this, the synthesis function $\mathcal{M}(s_{j,k})$ is wrapped in a localized exception handler. If the model triggers a token-overflow runtime error, the system catches the exception and triggers a recursive split of $s_{j,k}$ into $s_{j,k,\text{left}}$ and $s_{j,k,\text{right}}$. The resulting outputs are concatenated to form the complete waveform

$$w_{j,k} = \mathcal{M}(s_{j,k,\text{left}}) \parallel \mathcal{M}(s_{j,k,\text{right}}) \tag{3}$$

where $\parallel$ denotes signal concatenation.

4.2.1 Temporal Calculation and Synchronization

Since the TTS engine processes each sentence independently, the resulting audio segments must be concatenated into a continuous chapter stream that maintains natural prosody without artifacts. Let $\tau(w)$ denote the duration of a waveform in seconds. To mitigate boundary artifacts (such as digital clicks or hallucinated breath sounds common in flow-matching models), a linear fade-out filter is applied to the terminal segment (e.g., 50 ms) of each waveform. Subsequently, to ensure distinct audio separation between sentences, a silence padding δ (typically 0.15 s) is appended.

The cumulative audio stream A_j for a chapter j is formed by the sequential concatenation of these processed waveforms

$$A_j = (w'_{j,1} \parallel \delta) \parallel (w'_{j,2} \parallel \delta) \parallel \cdots \parallel (w'_{j,N_j} \parallel \delta) \tag{4}$$

where $w'_{j,k}$ represents the fade-out processed waveform.

The synchronization timestamps for the SMIL files are derived deterministically from the construction in Equation (4). Let $t_{j,k,\text{start}}$ and $t_{j,k,\text{end}}$ represent the clip begin and clip end times for sentence $s_{j,k}$. The start time is defined as the summation of all previous audio durations

$$t_{j,k,\text{start}} = \sum_{i=1}^{k-1} (\tau(w'_{j,i}) + \tau(\delta)) \tag{5}$$

To achieve a "gapless" visual experience, where the highlighting remains active during the silence δ between sentences, we extend the active interval of the current sentence to meet the start of the subsequent sentence

$$t_{j,k,\text{end}} = t_{j,k+1,\text{start}} \tag{6}$$

For the final sentence N_j, the end time is its natural duration $t_{j,N_j,\text{end}} = t_{j,N_j,\text{start}} + \tau(w'_{j,N_j}) + \tau(\delta)$. This gapless continuity prevents the visual "flicker" effect often observed in audio-e-books where the highlight disappears during pauses. Furthermore, strict adherence to contiguous time intervals is critical, as certain reading systems validation checks will reject Media Overlays containing temporal gaps or overlaps.

4.3 Phase 3: Packaging and Accessibility Integration

The final phase involves reconstructing the EPUB container to comply with EPUB 3.3 Accessibility specifications.

4.3.1 SMIL Generation

For each XHTML chapter, a corresponding SMIL file is generated mapping the structural IDs ($id_{j,k}$) to the time intervals calculated in Step 2. References are constructed using relative paths to ensure compatibility across reading systems:

```
<par id="par_f001">
   <text src="../text/chapter.xhtml#f001"/>
   <audio src="../audio/chapter.mp3"
         clipBegin="t_start" clipEnd="t_end"/>
</par>
```

4.3.2 Visual Accessibility (Dark Mode)

To ensure the visual overlay remains legible across various reading environments, we inject an additional CSS rule utilizing the `prefers-color-scheme` media query. This dynamically adapts the highlight color (e.g., high-contrast yellow for Light Mode, muted purple for Dark Mode) based on the user's system preferences.

4.3.3 Metadata and Packaging

Finally, the Open Packaging Format (OPF) file is updated. We inject the `media:duration` metadata for every SMIL file and the total publication duration, calculated by summing the precise durations derived in Step 2. The `media:active-class` metadata is inserted to trigger the CSS highlighting mechanism. The resulting assets are zipped to produce the final validated EPUB 3 file with Media Overlays.

5 Implementation and Usage

The proposed framework is implemented as an open-source MIT Licenced Python library, designed for deployment on consumer-grade hardware (Linux/WSL, macOS, or Windows) with optional GPU acceleration.

5.1 Installation

The repository supports the two TTS systemt *Chatterbox* and *XTTS-v2*, see Section 3.

5.1.1 Environment Setup

We recommend initializing a Python 3.11 virtual environment to manage the dependencies. This isolation is critical for handling the specific version constraints for `numpy` and `torch` mandated by the underlying audio processing libraries:

```
# Clone the repository
git clone https://github.com/hugohammer/TTS-Narrated-Ebook-Creator.git
cd TTS-Narrated-Ebook-Creator
# Create and activate a virtual environment. It is
# recommeneded to use separate environments for the
# two TTS models.
python3.11 -m venv env_chatterbox
source env_chatterbox/bin/activate
python3.11 -m venv env_xtts
source env_xtts/bin/activate
```

5.1.2 Dependencies

The project relies on several core libraries: `EbookLib` for parsing the EPUB container, `BeautifulSoup4` for DOM manipulation, and `FFmpeg` for audio signal processing. The backend-specific dependencies can be installed via the provided requirement files:

```
# For Chatterbox
pip install -r requirements/requirements_chatterbox.txt

# For XTTS-v2
pip install -r requirements/requirements_xtts.txt
```

5.2 Execution

The pipeline is executed via a Command Line Interface. The user must provide the source EPUB file and a reference audio sample (approximately 15 seconds of clean speech in WAV format) to define the narrator's voice profile.

5.2.1 Running the Scripts

To execute the creation process using the Chatterbox engine:

```
python src/epub_processor_chatterbox.py input_book.epub \
     --voice assets/neutral_narrator.wav \
     --output output_book.epub
```

and for XTTS-v2

```
python src/epub_processor_xtts.py input_book.epub \
     --voice assets/neutral_narrator.wav \
     --output output_book.epub
```

5.2.2 Configuration Arguments

The tool supports several runtime arguments to customize the output:

- `--gpu`: Forces the use of CUDA acceleration if available, significantly reducing synthesis time.
- `--language [code]`: Specifies the target language for the TTS engine (default: 'en').
- `--skip-audio`: Enables a debug mode that created the EPUB without running the computationally expensive TTS step.

Upon completion, the script outputs a validated EPUB 3 file containing the synchronized Media Overlays. The file can be uploaded to e.g. the Thorium Reader for Windows, MacOS and Linux or the BookFusion or Storyteller apps for Android/iOS.

6 Evaluation of Alignment Strategies

As discussed, a benefit of our implementation was that it ensured perfect synchronization between text highlighting and audio. However, from an implementation point of view, it could be argued that using existing forced alignment methods would have been easier, since TTS and alignment could have been separated during the creation of the narrated e-book. To analyse the potential of using forced alignment, we considered two recent methods: Afaligner [17] and Storyteller [21]. Afaligner was the forced alignment method used in Syncabook [16], was inspired by Aeneas, and was based on the Dynamic Time Warping algorithm [20]. Storyteller employed a transcription-based alignment strategy. First, the audio was transcribed into time-stamped text using Whisper [18]. This transcription was then mapped to the e-book's content using fuzzy string matching to resolve discrepancies between what was narrated in the audio and the source text.

The performance of the alignment methods was evaluated on the e-book *The Gift of the Magi*, freely available from the Gutenberg Project [14]. The forced alignment methods received as input the e-book and the TTS audio obtained from Chatterbox and produced as output an EPUB 3 Media Overlay. The .epub file was further unzipped, and the time stamps of the SMIL files were compared against the exact time stamps obtained using our method described in Section 4.2. The Storyteller method did not use the same sentence tokenizer as our method, and therefore some sentences were potentially different. We employed a filtering strategy where the time stamps were only compared for sentences that were identical and in the same chapter (85% of the sentences).

Table 2 shows key statistics for the distribution of the drift for the different sentences for the two forced alignment methods. Figure 3 shows histograms of the drift distributions. Please note that the scale on the x-axis is different for the three histograms. A negative value means that the text highlighting lagged behind the audio for the forced alignment method, while a positive value means that the text highlighting appeared before the audio. Readers are more sensitive to text highlighting lagging behind the audio than the other way around. Even a small lag of 50 ms could can affect a reading experience,

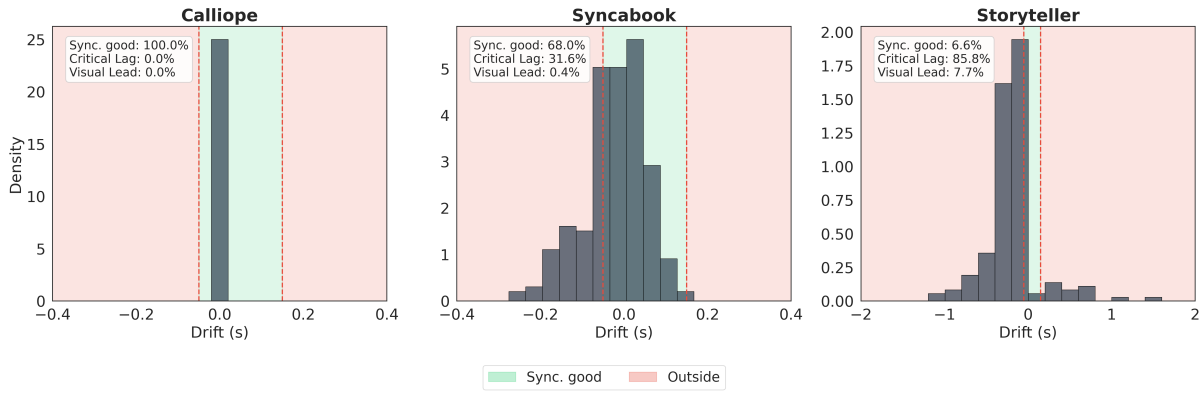


Figure 3: Histograms of the drift distributions for the forced alignment methods. Note that the scale on the x axis is different for the different figures. The green and red areas in the figures, shows where the drift is acceptable or where it may affect the reading experience. Our method is exact, and the drift was zero.

Table 2: Key statistics for the synchronization drift distribution in seconds for our method and the forced alignment methods used in Syncabook and Storyteller. P10 and P90 refer to the 10% and 90% quantiles of the drift distributions, respectively.

Method	Min	P10	Mean	Median	P90	Max
Calliope	0	0	0	0	0	0
Storyteller	-34.91	-0.75	-1.11	-0.20	-0.01	1.60
Syncabook	-0.96	-0.14	-0.03	-0.01	0.07	0.49

while the text highlighting could typically be as much as 150 ms early before affecting the reading experience [3, 8]. The green and red areas in the figure show the intervals where the drift was within and outside the acceptable interval, $[-50, 150]$ ms, respectively.

In Table 2, we see that the Storyteller forced alignment ran into substantial synchronization issues for some sentences, with drift of over 30 seconds. Syncabook was fairly consistent, with a maximal drift of 0.96 seconds. However, in the middle panel of Figure 3, we see that for over 30% of the sentences, the lag was more than the delay of 50 ms and can affect the reading experience. For Storyteller, the synchronization was satisfactory for only 6.6% of the sentences. Since our method does not rely on forced alignment, the drift was zero as shown in the upper row of Table 2 and the left panel of Figure 3.

The main takeaway is that even when the forced alignment algorithm worked as expected (Syncabook), the resulting drift could still represent an issue for the reader. This motivates the use of the exact alignment strategy developed in this paper.

7 Conclusion

In this work, we have presented a robust, open-source framework using SOTA open source TTS to convert standard text-based e-books into high quality narrated e-books in the EPUB 3 Media Overlays format. To the best of our knowledge this is the first open source solution to perform this task. Our solution offers several innovative steps. First, by using the exact timings from the TTS, resulting in perfect synchronization between text highlighting and audio. Our experiments showed that alternatively using forced alignment resulted in drift between audio and text significant enough to degrade the reading experience. Second, the surgical insertions strategy ensures that the original publisher's typography, layout, and embedded media were preserved. Finally, the implementation operates entirely offline, utilizing the local neural TTS models Chatterbox and XTTS-v2. This approach not only eliminates the costs of commercial APIs but also resolves privacy concerns and copyright compliance issues associated

with uploading protected content to third-party cloud services.

A first natural step for future research is to add a graphical user interface, for the installation and the usage, to make the method easier accessible for more users. It is also interesting to explore quantized TTS models to evaluate the potential for using the method directly on mobile devices.

References

[1] aedocw. epub2tts: Turn an epub or text file into an audiobook. https://github.com/aedocw/epub2tts, 2024. GitHub Repository.

[2] Edresson Casanova, Kelly Davis, Eren Gölge, Görkem Göknar, Iulian Gulea, Logan Hart, Aya Aljafari, Joshua Meyer, Reuben Morais, Samuel Olayemi, et al. Xtts: a massively multilingual zero-shot text-to-speech model. *arXiv preprint arXiv:2406.04904*, 2024.

[3] Brianna L Conrey and David B Pisoni. Audiovisual asynchrony detection for speech and nonspeech signals. In *AVSP*, volume 2003, pages 25–30, 2003.

[4] DAISY Consortium. Daisy standard overview. https://daisy.org/activities/standards/daisy/, 2025.

[5] EDRLab. Thorium Reader. https://www.edrlab.org/software/thorium-reader/, 2025.

[6] ElevenLabs. Elevenreader official website. https://elevenreader.io/, 2025.

[7] Funway. audible-epub3-maker: Generate audiobooks from plain epub files. https://github.com/funway/audible-epub3-maker, 2025. GitHub Repository.

[8] Ken W Grant and Steven Greenberg. Speech intelligibility derived from asynchronous processing of auditory-visual information. In *AVSP*, volume 200, pages 132–137, 2001.

[9] Jookyoung Jung and Wenrui Zhang. The impact of text-audio synchronized enhancement on collocation learning from reading-while-listening: an extended replication of. *International Review of Applied Linguistics in Language Teaching*, 63(3):2201–2229, 2025.

[10] Jaehyeon Kim, Jungil Kong, and Juhee Son. Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. In *International Conference on Machine Learning*, pages 5530–5540. PMLR, 2021.

[11] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *11th International Conference on Learning Representations, ICLR 2023*, 2023.

[12] Mozilla Developer Network. Web speech api documentation. https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API/Using_the_Web_Speech_API, 2025.

[13] NaturalSoft Ltd. Naturalreader official website. https://www.naturalreaders.com/, 2025.

[14] O. Henry. The Gift of the Magi. Project Gutenberg, January 2005. eBook #7256. Updated: Dec 24, 2021.

[15] Alberto Pettarin. aeneas: A python/c library and set of tools to synchronize audio and text. https://github.com/readbeyond/aeneas, 2016.

[16] r4victor. syncabook: A tool for creating ebooks with synchronized text and audio. https://github.com/r4victor/syncabook, 2024. GitHub Repository.

[17] r4victor. Afaligner: A tool for forced alignment. https://github.com/r4victor/afaligner/, 2025.

[18] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *International conference on machine learning*, pages 28492–28518. PMLR, 2023.

[19] Resemble AI. Chatterbox-TTS. https://github.com/resemble-ai/chatterbox, 2025. GitHub repository.

[20] Hiroaki Sakoe and Seibi Chiba. Dynamic programming algorithm optimization for spoken word recognition. *IEEE transactions on acoustics, speech, and signal processing*, 26(1):43–49, 1978.

[21] Smoores. Storyteller: Self-hosted platform for syncing audiobooks and ebooks. https://gitlab.com/storyteller-platform/storyteller, 2025. GitLab Repository.

[22] World Wide Web Consortium. Epub 3.3 specification. https://www.w3.org/TR/epub-33/, 2025.